```
    {
      "_index":"international_movies",    <-- Index 2
      "_id":22
    },
    {
      "_index":"top100_movies",    <-- Index 3
      "_id":33
    }
  ]
}
```

The request is formed with the requirement of fetching three documents from three different indexes: `classic_movies`, `international_movies`, and `top100_movies`. Note that if the index doesn't exist, we will get an `index_not_found_exception`.

We can provide as many indexes as required, although this API has a downside: we have to create an individual `_index`/`_id` pair for each ID. Unfortunately, Elasticsearch does not yet allow the `_id` attribute to accept an array of IDs. We can hope that the Elastic folks implement this feature in the near future.

5.2.3 The ids query

We looked at using the `_mget` API to fetch multiple documents in the previous section. However, there is another route to fetch multiple documents: using an `ids` (short for IDs) query. This simple `search` query takes a set of document IDs to return the documents. The `ids` query is available as part of the search APIs. We discuss this API in detail in chapters 8 to 10, but here's the query in action if you are curious.

Listing 5.11 Using an `ids` query to fetch multiple documents

```
GET classic_movies/_search
{
  "query": {
    "ids": {
      "values": [1,2,3,4]
    }
  }
}
```

We can also fetch from multiple indexes by adding the indexes to the URL. Here's an example:

```
GET classic_movies,international_movies/_search
{
  # body
}
```

That wraps up how to retrieve multiple documents from a single or multiple indexes. Now, let's move our focus to the responses. Did you notice that our responses (see figure 5.6) get metadata along with the original source document? What if we want to fetch only the source without metadata? Or what if we want to hide some of the sensitive